Contents

Script thuyết trình — Spring Batch: xử lý hàng triệu bản ghi an toàn **1**
Slide 1 — Mở đầu . . . 1
Slide 2 — Đặt vấn đề: bài toán cộng lãi cuối tháng . . . 1
Slide 3 — Vấn đề khi dùng vòng lặp for thuần . . . 2
Slide 4 — Spring Batch giải quyết 4 rắc rối đó . . . 2
Slide 5 — Kiến trúc tổng thể . . . 2
Slide 6 — Dây chuyền 3 khâu: Reader, Processor, Writer . . . 7
Slide 7 — Job và Step . . . 8
Slide 8 — Chunk: chia để trị có chọn lọc . . . 8
Slide 9 — Sơ đồ trình tự xử lý Chunk . . . 8
Slide 10 — Sổ ghi chép: JobRepository . . . 8
Slide 11 — Trạng thái và số liệu theo dõi . . . 9
Slide 12 — Khi gặp sự cố: Skip, Retry, Restart . . . 9
Slide 13 — Cài đặt và khởi tạo dự án . . . 9
Slide 14 — Cấu hình Job, Step và ItemProcessor — đi vào luồng gọi hàm thật . . . 10
Slide 15 — Khởi chạy Job và đọc nhật ký . . . 12
Slide 16 — Lưu đồ chống chạy trùng và Restart . . . 12
Slide 17 — Mã nguồn triển khai thực tế . . . 13
Slide 18 — Demo: nút “Chọn ngay” làm gì? . . . 13
Slide 19 — Khi nào nên dùng, khi nào không nên . . . 14
Slide 20 — Tóm tắt . . . 14

Script thuyết trình — Spring Batch: xử lý hàng triệu bản ghi an toàn

> Văn nói, chia theo từng slide. Dự án demo: `spring-batch-example` — job `selectingWeeklyWinner` chọn người thắng cuộc hằng tuần, tích hợp PostgreSQL và Kafka, kèm dashboard React.

Slide 1 — Mở đầu

Chào mọi người, hôm nay mình xin trình bày về Spring Batch, với chủ đề là “xử lý hàng triệu bản ghi một cách an toàn — cách hoạt động và khi nào nên dùng trong thực tế”.

Để mọi người dễ hình dung, mình sẽ đi kèm luôn một dự án demo thật sự đang chạy ở đây, đó là một hệ thống chọn “người thắng cuộc” hằng tuần cho các khách hàng có giao dịch vượt ngưỡng. Toàn bộ lý thuyết trên slide đều có code thật trong dự án này, nên cuối phần mình sẽ demo trực tiếp luôn.

Slide 2 — Đặt vấn đề: bài toán cộng lãi cuối tháng

Vậy đầu tiên, batch là gì? Mình lấy một ví dụ rất đời thường: mỗi cuối tháng, ngân hàng phải cộng lãi cho 5 triệu tài khoản.

Nếu nghĩ đơn giản thì việc này dễ ợt, đúng không? Chỉ cần lặp qua từng tài khoản, tính số tiền lãi theo số dư, rồi cộng tiền vào thôi. Nhưng nhìn kỹ thì đây là một loại công việc đặc biệt, nó có ba dấu hiệu nhận dạng.

Thứ nhất, nó chạy định kỳ, không trả lời ngay. Mỗi tháng chỉ làm một lần, vào nửa đêm, khách hàng không đứng đợi kết quả. Thứ hai, dữ liệu cực lớn, nhiều gigabyte, không thể tải hết vào bộ nhớ. Thứ ba, ưu tiên độ chính xác tuyệt đối, cộng thiếu hay cộng dư một đồng cũng là sự cố.

Loại công việc xử lý dữ liệu lớn theo lô, định kỳ như thế này, người ta gọi là Batch. Trong dự án demo của mình cũng đúng bản chất đó: job chạy hằng ngày lúc 00:01:30, không ai đứng đợi trước màn hình.

Slide 3 — Vấn đề khi dùng vòng lặp for thuần

Giờ giả sử anh dev mới vào nghề, viết một vòng lặp for thôi, chạy thử trên 5 triệu tài khoản, thì sẽ gặp ngay bốn vấn đề.

Một là quá tải bộ nhớ. Nạp 5 triệu tài khoản vào RAM thì máy chủ lập tức sập, lỗi OutOfMemory.

Hai là dữ liệu hỏng. Chỉ cần một tài khoản có số liệu lỗi, cả vòng lặp bị ngắt, dư nächsten... à nhầm, dừng luôn từ đó.

Ba là mất điện, sự cố giữa chừng. Đang chạy tới tài khoản thứ 3 triệu thì cúp điện, chạy lại từ đầu thì 3 triệu người đầu bị cộng lãi hai lần. Đây là thảm họa thật sự.

Và bốn, mù thông tin. Chương trình chạy ngầm cả tiếng, mình không biết nó đang chạy tới đâu, xong bao nhiêu, còn bao lâu, và lỗi nằm ở dòng nào.

Bốn rắc rối này chính là lý do Spring Batch ra đời.

Slide 4 — Spring Batch giải quyết 4 rắc rối đó

Spring Batch là framework tiêu chuẩn của hệ sinh thái Spring, chuyên biệt cho xử lý dữ liệu hàng loạt. Nó có sẵn bộ công cụ cho đúng 4 rắc rối vừa nói.

Với việc dữ liệu nhiều, RAM quá tải, nó xử lý theo từng nhóm nhỏ, gọi là Chunk Processing. Với bản ghi hỏng dừng cả đợt, nó tự động bỏ qua và ghi nhật ký lỗi, gọi là Skip. Với mất điện giữa chừng, nó ghi nhớ mốc an toàn và chạy tiếp mượt mà, gọi là Restart. Còn với việc không biết tiến độ, nó có một cuốn sổ ghi chép chi tiết tự động lưu vào database, gọi là JobRepository.

Trong dự án demo, mình dùng Spring Batch 6.0.6 — đúng phiên bản mới nhất hiện nay — trên Spring Boot 4.1.2 và Java 21.

Slide 5 — Kiến trúc tổng thể

1. Giới thiệu tổng quan

“Ok, thì ở đây em sẽ đi qua tổng quan về kiến trúc của Spring Batch Framework và cách một batch job được thực thi từ lúc bắt đầu cho tới khi hoàn thành.

Nhìn tổng thể sơ đồ thì mình có thể chia thành các phần chính như sau.

Đầu tiên là **bộ kích hoạt Batch**, tiếp theo là **JobLauncher**, sau đó là phần xử lý chính gồm **Job, Step, ItemReader, ItemProcessor và ItemWriter**.

Bên dưới là **JobRepository**, có nhiệm vụ quản lý và lưu trạng thái thực thi của toàn bộ quá trình.

Cụ thể

2. Bước 1 – Job Scheduler / CLI kích hoạt Batch

"Đầu tiên, ở phía bên trái sơ đồ là **Job Scheduler hoặc CLI**.

Đây là nơi khởi tạo quá trình chạy batch.

Thì thực tế mình có thể trigger batch bằng nhiều cách khác nhau.

Ví dụ như dùng `@Scheduled` để cấu hình chạy tự động theo lịch, chẳng hạn mỗi ngày lúc 12 giờ đêm.

Hoặc có thể chạy thủ công thông qua CommandLineJobRunner, hay gọi từ một Admin REST API.

Nói chung, phần này chỉ đóng vai trò là **điểm kích hoạt**, chứ chưa trực tiếp xử lý dữ liệu.

Sau khi nhận được yêu cầu chạy, hệ thống sẽ chuyển sang JobLauncher."

3. Bước 2 – JobLauncher khởi chạy Job

"Tiếp theo là **JobLauncher**.

Thì JobLauncher có nhiệm vụ khởi chạy một Job cụ thể.

Khi gọi `run()`, mình sẽ truyền vào hai thành phần chính.

Một là Job cần thực thi.

Hai là JobParameters, tức là những tham số đi kèm với lần chạy đó.

Ví dụ như ngày xử lý dữ liệu, mã batch, hoặc một số tham số phục vụ cho logic nghiệp vụ.

Ở bước này, Spring Batch sẽ phối hợp với JobRepository để quản lý thông tin của lần chạy.

Cụ thể là xác định JobInstance tương ứng với Job và JobParameters, đồng thời tạo JobExecution để theo dõi lần thực thi hiện tại.

Trạng thái ban đầu sẽ được ghi nhận, và khi bắt đầu thực thi thì JobExecution sẽ chuyển sang trạng thái STARTED.

Một điểm cần lưu ý là **JobInstance và JobExecution không giống nhau**.

JobInstance đại diện cho một lần chạy logic, được xác định dựa trên Job và JobParameters có tính định danh.

Còn JobExecution đại diện cho một lần thực thi thực tế của JobInstance đó.

Ví dụ cùng một JobInstance, nếu lần chạy đầu tiên bị lỗi và mình restart lại, thì có thể phát sinh một JobExecution mới.

Sau khi hoàn tất các bước khởi tạo, JobLauncher sẽ bắt đầu thực thi Job."

4. Bước 3 – Job điều phối các Step

“Tiếp theo là thành phần **Job**.

Mọi người có thể hiểu đơn giản Job chính là đơn vị công việc tổng thể của Spring Batch.

Một Job sẽ bao gồm một hoặc nhiều Step.

Ví dụ trong một bài toán xử lý dữ liệu tài khoản, mình có thể chia thành ba Step.

Step đầu tiên đọc và kiểm tra dữ liệu đầu vào.

Step thứ hai xử lý nghiệp vụ.

Và Step thứ ba tổng hợp hoặc xuất kết quả.

Tùy vào thiết kế, các Step có thể được thực hiện tuần tự hoặc theo flow điều kiện.

Trong sơ đồ này, em đang tập trung vào trường hợp một Step xử lý dữ liệu theo mô hình Chunk.

Khi Job chạy đến Step tương ứng, Spring Batch sẽ bắt đầu thực thi phần xử lý bên trong Step đó.”

5. Bước 4 – Step thực thi theo Chunk-Oriented Processing

“Đến phần quan trọng nhất của sơ đồ, đó là **Step**.

Trong Spring Batch thì Step là đơn vị thực thi cụ thể.

Có hai cách triển khai phổ biến.

Một là Tasklet, thường dùng cho các tác vụ đơn giản, ví dụ xóa file, gọi API hoặc thực hiện một thao tác đơn lẻ.

Hai là Chunk-Oriented Processing, thường dùng để xử lý lượng dữ liệu lớn.

Ở sơ đồ này, em đang sử dụng mô hình Chunk.

Thì thay vì đọc toàn bộ dữ liệu vào memory rồi xử lý một lần, Spring Batch sẽ chia dữ liệu thành từng nhóm nhỏ, gọi là Chunk.

Ví dụ mình có 5 triệu tài khoản cần xử lý và cấu hình chunk size bằng 1.000.

Thì hệ thống sẽ đọc và xử lý dữ liệu theo từng nhóm, tối đa 1.000 bản ghi trong một Chunk.

Mỗi Chunk sẽ được xử lý trong một transaction riêng.

Bây giờ em sẽ đi vào ba thành phần chính bên trong Step.”

6. Bước 5 – ItemReader đọc dữ liệu đầu vào

“Đầu tiên là **ItemReader**.

ItemReader chịu trách nhiệm đọc dữ liệu từ nguồn đầu vào.

Nguồn này có thể là database, file CSV, XML hoặc một hệ thống dữ liệu khác.

Ví dụ trong trường hợp này, mình có 5 triệu tài khoản nằm trong database.

Thì ItemReader sẽ lần lượt đọc từng bản ghi và trả dữ liệu về cho Spring Batch.

Một điểm cần chú ý là ItemReader thường trả về từng item thông qua phương thức `read()`.

Spring Batch sẽ gọi `read()` nhiều lần để thu thập đủ dữ liệu cho một Chunk.

Ví dụ chunk size bằng 1.000 thì framework sẽ lần lượt đọc tối đa 1.000 item để đưa sang bước xử lý tiếp theo.

Khi hết dữ liệu, phương thức `read()` sẽ trả về null, lúc này Spring Batch hiểu rằng không còn item nào cần xử lý.

Nếu Reader hỗ trợ lưu trạng thái, ví dụ thông qua ItemStream, thì vị trí đọc cũng có thể được lưu vào ExecutionContext.

Thông tin này sẽ phục vụ cho việc restart khi Job bị lỗi giữa chừng."

7. Bước 6 – ItemProcessor xử lý nghiệp vụ

"Sau khi đọc dữ liệu xong thì các item sẽ được chuyển qua **ItemProcessor**.

Đây là nơi thực hiện logic nghiệp vụ.

Ví dụ mình có thể kiểm tra tài khoản có hợp lệ hay không, chuẩn hóa thông tin, tính toán dữ liệu hoặc chuyển đổi từ object đầu vào sang object đầu ra.

Trong Spring Batch, ItemProcessor sẽ xử lý từng item thông qua phương thức `process()`.

Ví dụ Reader đọc được một tài khoản, thì Processor sẽ kiểm tra trạng thái tài khoản đó.

Nếu hợp lệ thì trả về dữ liệu đã xử lý.

Nếu không muốn đưa item đó sang bước ghi, Processor có thể trả về null để filter item.

Ngoài ra, nếu trong quá trình xử lý phát sinh exception thì tùy theo cấu hình, Spring Batch có thể thực hiện retry, skip hoặc đánh dấu Step thất bại.

Sau khi các item được xử lý, những item hợp lệ sẽ được gom lại để chuyển sang ItemWriter."

8. Bước 7 – ItemWriter ghi dữ liệu đầu ra

"Tiếp theo là **ItemWriter**.

ItemWriter chịu trách nhiệm ghi dữ liệu sau khi đã được xử lý.

Ví dụ mình có thể ghi xuống PostgreSQL, Redis, file hoặc một hệ thống bên ngoài.

Điểm khác biệt so với Reader và Processor là Writer thường nhận một Chunk gồm nhiều item để thực hiện ghi theo lô.

Ví dụ chunk size bằng 1.000, sau khi đọc và xử lý đủ số lượng item tương ứng, Spring Batch sẽ gọi `write()` với Chunk chứa các item được giữ lại sau bước Processor.

Nếu mình sử dụng JDBC Batch thì có thể gom nhiều câu lệnh insert hoặc update để giảm số lần tương tác với database.

Điều này đặc biệt hữu ích khi xử lý lượng dữ liệu lớn.

Sau khi Writer thực hiện thành công và transaction được commit, Spring Batch sẽ tiếp tục xử lý Chunk tiếp theo.

Quá trình Reader, Processor và Writer sẽ lặp lại liên tục cho tới khi không còn dữ liệu đầu vào.”

9. Bước 8 – JobRepository lưu trạng thái và hỗ trợ Restart

“Bây giờ em sẽ nói đến phần bên dưới sơ đồ, đó là **JobRepository**.

Đây là thành phần rất quan trọng của Spring Batch.

JobRepository có nhiệm vụ quản lý metadata của quá trình thực thi Job và Step.

Ví dụ như Job nào đang chạy, Step nào đã hoàn thành, Step nào bị lỗi và số lượng dữ liệu đã được xử lý.

Trong quá trình thực thi, Spring Batch sẽ cập nhật các thông tin như readCount, writeCount, skipCount và commitCount.

Ngoài ra còn có JobExecutionContext và StepExecutionContext.

Hai thành phần này được sử dụng để lưu trạng thái cần thiết phục vụ cho quá trình thực thi và restart.

Ví dụ một Job đang xử lý 5 triệu bản ghi.

Sau khi xử lý được 3 triệu bản ghi thì hệ thống gặp lỗi.

Nếu Reader và các thành phần liên quan được thiết kế hỗ trợ restart đúng cách, Spring Batch có thể sử dụng trạng thái đã lưu trong ExecutionContext để tiếp tục từ checkpoint phù hợp.

Như vậy mình không nhất thiết phải xử lý lại toàn bộ 5 triệu bản ghi từ đầu.

Tuy nhiên, việc restart chính xác còn phụ thuộc vào cách triển khai Reader, Writer, transaction và cấu hình của Job.

Một điểm nữa là JobRepository không trực tiếp lưu dữ liệu nghiệp vụ.

Nó chủ yếu lưu metadata phục vụ cho việc quản lý quá trình thực thi Batch.”

10. Bước 9 – Database Metadata

“Tiếp theo, toàn bộ metadata này sẽ được lưu xuống database thông qua JDBC.

Ở phía dưới sơ đồ là nhóm các bảng metadata mặc định của Spring Batch.

Ví dụ như BATCH_JOB_INSTANCE để lưu thông tin JobInstance.

BATCH_JOB_EXECUTION để lưu các lần thực thi Job.

Và BATCH_STEP_EXECUTION để lưu trạng thái thực thi của từng Step.

Ngoài ba bảng này còn có các bảng lưu JobParameters và ExecutionContext.

Tổng cộng Spring Batch có sáu bảng metadata chính, bên cạnh các bảng sequence hoặc thành phần hỗ trợ tùy database.

Nhờ những bảng này, mình có thể truy vấn lịch sử thực thi, kiểm tra trạng thái Batch và xác định nguyên nhân khi một Job gặp lỗi.

Đồng thời, đây cũng là nền tảng hỗ trợ cơ chế restart của Spring Batch.”

11. Tổng kết toàn bộ Flow

“Ok, thì em sẽ tổng kết lại toàn bộ flow một lần nữa.

Đầu tiên, Scheduler hoặc CLI sẽ trigger Batch.

Sau đó JobLauncher nhận yêu cầu và khởi chạy Job cùng với JobParameters.

Spring Batch sẽ tạo hoặc xác định JobInstance tương ứng, tạo JobExecution và ghi nhận trạng thái thực thi thông qua JobRepository.

Tiếp theo, Job sẽ điều phối các Step.

Ở mỗi Step sử dụng Chunk-Oriented Processing, ItemReader sẽ đọc dữ liệu, ItemProcessor xử lý nghiệp vụ và ItemWriter ghi kết quả.

Quá trình này được thực hiện lặp lại theo từng Chunk.

Sau mỗi Chunk được commit thành công, Spring Batch sẽ cập nhật trạng thái và các thông tin cần thiết vào JobRepository.

Khi toàn bộ dữ liệu được xử lý thành công, Step sẽ hoàn thành, sau đó Job cũng được đánh dấu COMPLETED.

Còn nếu xảy ra lỗi không được xử lý, Step hoặc Job có thể chuyển sang trạng thái FAILED.

Dựa trên metadata đã lưu, mình có thể kiểm tra nguyên nhân lỗi và thực hiện restart nếu Job đáp ứng các điều kiện restart.

Như vậy, **điểm mạnh của Spring Batch không chỉ nằm ở việc xử lý dữ liệu theo từng Chunk, mà còn ở khả năng quản lý transaction, theo dõi trạng thái, xử lý lỗi và hỗ trợ restart**.

Đây là những yếu tố rất quan trọng đối với các hệ thống cần xử lý khối lượng dữ liệu lớn.

Đó là tổng quan về flow hoạt động của Spring Batch Framework mà em muốn trình bày.”

Cái hay của dự án demo này là nó có đủ cả hai. Phần batch là job chọn người thắng, chạy ngầm. Phần ứng dụng thường ngày là một dashboard web, làm giao diện điều khiển và xem kết quả. Và hai luồng này gặp nhau ở Kafka và PostgreSQL: batch chạy xong bắn kết quả lên Kafka, dashboard đọc từ database và API để hiển thị.

Slide 6 — Dây chuyền 3 khâu: Reader, Processor, Writer

Đi vào bên trong, mỗi đơn vị xử lý của Spring Batch là một dây chuyền ba khâu, giống một mô hình nhà máy đóng gói.

Khâu một là ItemReader, coi như người lấy hàng từ kho. Nó đọc từng bản ghi, thường phân trang tự động theo từng khối. Khâu hai là ItemProcessor, người kiểm tra và dán nhãn, tức là biến đổi dữ liệu, tính toán, hoặc loại bỏ bản ghi không hợp lệ. Khâu ba là ItemWriter, người xếp hàng vào kho thành phẩm, ghi trọn vẹn cả lô trong một giao dịch.

Trong dự án demo, ba step của job đều theo đúng mô hình này. Ví dụ step nạp người dùng: Reader gọi REST API qua WebClient, Processor chuyển DTO thành entity, Writer ghi vào PostgreSQL bằng bulk writer.

Slide 7 — Job và Step

Tiếp theo là hai khái niệm cốt lõi: Step là một dây chuyền, còn Job là cả nhà máy.

Một nhà máy gồm nhiều dây chuyền chạy nối tiếp. Step trước hoàn tất thành công thì Step sau mới được kích hoạt, để đảm bảo an toàn tuyệt đối.

Job demo của mình tên là `selectingWeeklyWinner`, được khai báo trong file `BatchConfiguration.java`. Nó gồm ba step nối tiếp nhau:

Step một, `fetchUserData`: nạp danh sách người dùng từ API và lưu vào database. Step hai, `insertUserTransactions`: đọc giao dịch từ file CSV và ghi vào bảng transactions. Step ba, `selectLuckyWinner`: lọc ra những người có tổng giao dịch tuần qua vượt ngưỡng 100, chọn ngẫu nhiên một người, rồi gửi kết quả lên Kafka.

Quy tắc an toàn ở đây rất rõ: nếu step một lỗi, step hai và ba sẽ không chạy, tránh gửi kết quả sai hoặc lưu dữ liệu bẩn.

Slide 8 — Chunk: chia để trị có chọn lọc

Giờ đến trái tim của Spring Batch, đó là Chunk processing. Mình ví dụ chấm thi cho dễ hiểu: cô giáo chia 500 bài thi thành từng xấp 20 bài. Chấm xong mỗi xấp thì ghi điểm ngay vào sổ. Nếu có sự cố, cô chỉ cần chấm lại xấp đang dở, không phải chấm lại từ đầu.

Cơ chế chunk là vậy: Reader đọc từng bản ghi cho đủ N mục, Processor nhận từng mục và xử lý, rồi Writer nhận trọn vẹn N mục đã xử lý, commit một giao dịch duy nhất cho cả xấp. Mỗi xấp hoàn tất là một mốc an toàn được lưu chắc.

Về kích thước chunk: chọn quá nhỏ thì lưu liên tục, tốn công mở giao dịch. Chọn quá lớn thì tốn RAM, và nếu lỗi phải làm lại nhiều. Thực tế người ta hay chọn từ 20 đến 500.

Trong dự án, mình cấu hình chunk size bằng file properties luôn: step nạp user dùng chunk 10, step nhập giao dịch dùng chunk 100. Muốn đổi chỉ cần sửa một dòng trong `application-dev.properties`, không phải đụng code.

Slide 9 — Sơ đồ trình tự xử lý Chunk

Slide này tóm tắt một điểm tối ưu I/O rất quan trọng trong sơ đồ của ChunkOrientedTasklet: Reader và Processor chạy lặp từng bản ghi, nghĩa là N lần, nhưng Writer chỉ được gọi một lần duy nhất cho cả xấp N mục.

Cách làm này giảm hẳn số lần mở và commit giao dịch xuống database, mà vẫn giữ được tính nguyên vẹn của cả lô. Với 628 dòng giao dịch trong file CSV demo của mình, thay vì 628 lần commit, Spring Batch chỉ commit cỡ 7 lần với chunk 100.

Slide 10 — Sổ ghi chép: JobRepository

Spring Batch nhớ mình đã làm tới đâu bằng JobRepository, giống như cuốn sổ nhật ký công trường. Nó tự động ghi chi tiết mỗi lần chạy: đọc bao nhiêu bản ghi, ghi bao nhiêu, thành công

hay lỗi.

Các khái niệm trong sổ gồm: JobLauncher là nút bấm khởi chạy công việc, JobInstance là công việc cụ thể, ví dụ "tính lãi tháng 10", JobExecution là mỗi lần bấm máy, chạy lại là một execution mới của cùng instance, và JobRepository là cuốn sổ lưu tất cả vào các bảng BATCH_ trong database.

Và đây là điểm mình muốn nhấn mạnh với dự án demo: cái bảng lịch sử chạy trên dashboard đó, mình không hề tự viết thêm bất kỳ bảng log nào cả. Mọi thứ UI hiển thị đều đọc trực tiếp từ metadata mà Spring Batch ghi sẵn, thông qua `JobRepository` trong file `DashboardController.java`. Trong Spring Batch 6, interface `JobExplorer` cũ đã bị đánh dấu deprecated và thay thế chính thức bằng `JobRepository`, nên dự án mình dùng luôn API mới này. Đó chính là cái giá trị của JobRepository: framework trả sẵn cho mình phần "theo dõi và audit" mà nếu tự làm thì phải viết cả tuần.

Slide 11 — Trạng thái và số liệu theo dõi

Trạng thái chuẩn của Spring Batch gồm ba mức: STARTED, COMPLETED, FAILED. Trên dashboard, mình map ba trạng thái này thành "Đang chạy", "Hoàn tất", "Thất bại", kèm theo màu và hình dạng khác nhau để người dùng phân biệt ngay.

Số liệu theo dõi gồm bốn chỉ số: `readCount` là tổng số bản ghi đã đọc, `writeCount` là tổng số bản ghi ghi thành công, `commitCount` là số lần commit giao dịch, và `skipCount` là số bản ghi lỗi được bỏ qua an toàn.

Trên dashboard, mỗi lần chạy đều mở được xem chi tiết từng step với đủ 4 chỉ số này. Ví dụ khi demo các bạn sẽ thấy step nhập CSV có dòng "đọc 628, ghi 620, bỏ qua 8" — 8 dòng không khớp user nào đã được skip an toàn, không làm sập job.

Slide 12 — Khi gặp sự cố: Skip, Retry, Restart

Khi gặp sự cố, Spring Batch có ba cơ chế sẵn có.

Skip là bỏ qua: một tài khoản bị lỗi dữ liệu, hệ thống bỏ qua và ghi chép vào nhật ký, cả đợt 5 triệu người vẫn tiếp tục. Retry là thử lại: gọi dịch vụ ngoài bị nghẽn mạng, framework tự thử lại vài lần với độ trễ ngắn. Restart là chạy tiếp: mất điện ở tài khoản thứ 3 triệu, khi bật lại, hệ thống đọc mốc trong sổ và chạy tiếp từ xấp đã lưu.

Một lưu ý quan trọng: Restart chỉ an toàn khi bước ghi không tạo trùng lặp dữ liệu, tức là ghi theo khóa chính, kiểu cập nhật thay vì cộng dồn. Dự án demo của mình bám đúng nguyên tắc này: step nạp user dùng cơ chế upsert, user đã tồn tại thì merge cập nhật chứ không chèn thêm, nên chạy lại bao nhiêu lần cũng không sinh dữ liệu trùng.

Slide 13 — Cài đặt và khởi tạo dự án

Bây giờ sang phần thực hành. Để dựng dự án, mình dùng Spring Boot 4.1.2 với `spring-boot-starter-batch` — đây là cách khuyên dùng trong thực tế, nó tự động kết nối database và tạo

sẵn hệ thống bảng lưu vết BATCH_. Ngoài ra mình còn thêm `spring-boot-starter-batch-jdbc` để có sẵn `JobOperator` và hạ tầng batch chạy trên JDBC. Yêu cầu môi trường là Java 21.

Việc chạy được cả hệ thống thì mình cấu hình như thế nào? Toàn bộ hạ tầng nằm trong `docker-compose.yml`: một container PostgreSQL lưu dữ liệu và các bảng BATCH_, một container Kafka chạy chế độ KRaft để nhận kết quả, và container app.

Toàn bộ cấu hình môi trường được đặt trong file `.env`: cổng ứng dụng, cổng frontend, múi giờ, mật khẩu database, cổng Kafka. Mình cố tình đưa hết ra `.env` để máy nào chạy cũng đúng, không phải sửa code. File `.env.example` là bản mẫu, anh em chỉ cần copy sang `.env` và đặt mật khẩu mạnh vào.

Còn ở tầng ứng dụng, project chia ba profile: `prod` chạy trong Docker, `dev` chạy trên máy thật kết nối Docker, và `test` dùng H2 với embedded Kafka để chạy test không cần Docker gì cả. Profile nào cũng có file `application-<profile>.properties` riêng.

Vậy để chạy demo: `docker compose up -d postgres kafka` để bật hạ tầng, rồi chạy app bằng lệnh Maven với profile dev. Frontend thì `pnpm dev`, Vite sẽ tự proxy mọi request `/api` về cổng backend lấy từ `.env`. Khi lần đầu khởi động, PostgreSQL tự khởi tạo schema, Spring Batch tự tạo bảng BATCH_, Hibernate tự tạo bảng nghiệp vụ — mình không phải chạy một dòng SQL nào cả.

Slide 14 — Cấu hình Job, Step và ItemProcessor — đi vào luồng gọi hàm thật

Slide này ánh xạ trực tiếp sang code demo. Mình không cần annotation `@EnableBatchProcessing` như các bản cũ nữa, vì Spring Boot 4 tự động cấu hình hết: JobRepository, JobOperator, transaction manager.

Giờ mình muốn đi chậm lại một chút, đi theo đúng luồng gọi hàm trong code, từng file một, để các bạn thấy khi job chạy thì thực sự hàm nào gọi hàm nào.

Bước 1 — điểm bấm máy: file `src/main/java/com/ecom/winners/configurations/JobScheduler.jav`
Class này có một hàm `runSelectingWeeklyWinnerJob()`, gắn annotation `@Scheduled(cron = "30 1 0 * * *", zone = "${APP_TIME_ZONE:Asia/Ho_Chi_Minh}")` — nghĩa là Spring sẽ tự gọi hàm này hằng ngày lúc 00:01:30 theo múi giờ trong file `.env`. Trong hàm, nó tạo một object `Properties`, đặt `JobID` là timestamp hiện tại — tại sao cần tham số này thì slide sau mình giải thích — rồi gọi `jobOperator.start("selectingWeeklyWinner", params)`. `JobOperator` là bean Spring Boot tự cấp nhờ starter batch-jdbc, inject thẳng vào constructor của `JobScheduler`.

Một câu hỏi hay thường bị hỏi: tại sao tách `JobScheduler` ra khỏi file khai báo job? Vì circular dependency: `JobOperator` cần bean Job, mà `JobScheduler` lại cần inject `JobOperator`. Nếu viết cùng một class vừa khai báo Job vừa inject JobOperator thì Spring không giải nổi vòng phụ thuộc. Tách hai class là phá vòng tròn.

Bước 2 — lắp nhà máy: file `src/main/java/com/ecom/winners/configurations/BatchConfiguratio`
`jobOperator.start("selectingWeeklyWinner", ...)` sẽ tra bean Job có tên đó, và bean này được khai báo ở đây, trong hàm `selectingWeeklyWinner(...)`. Hàm này dùng `new JobBuilder("selectingWeeklyWinner", jobRepository)`, gắn `incrementer(new RunIdIncrementer())`, gắn listener, rồi `.start(fetchUserData).next(insertUserTransactions).next(s`

Ba cái tham số fetchUserData, insertUserTransactions, selectLuckyWinner chính là ba bean Step. Thứ tự trong chuỗi này chính là thứ tự chạy: step sau chỉ chạy khi step trước COMPLETED.

Bước 3 — Step 1, API → DB: file src/main/java/com/ecom/winners/configurations/UserData.java. Đây là step được gọi đầu tiên, mình đi kỹ hàm nó gọi thế nào. Trong class này có 4 bean, chạy theo dây chuyền reader → processor → writer.

Hàm thứ nhất, reader(...), có annotation @StepScope — nghĩa là mỗi lần chạy step thì tạo reader mới, để biến đếm trang luôn bắt đầu lại từ đầu. Reader này không tự viết, mà lắp bằng RestApiItemReaderBuilder trong file src/main/java/com/ecom/winners/readers/RestApiItemRea
Cách lắp như sau: RestApiItemReaderBuilder.<UserDTO>builder().webClient(WebClient.create
-> currentPage >= endPage).entity(UserDTO.class).mapper(objectMapper).build().
Host, path, page size, số trang đầu — số trang cuối tất cả đọc từ application-dev.properties. Khi job chạy, mỗi lần Spring Batch cần một "mục" mới, nó gọi hàm read() trong file builder này. Hàm read() làm đúng 3 việc: một, gọi endRead.test(page) — nếu trang hiện tại vượt trang cuối thì trả null, và trả null là tín hiệu Spring Batch hiểu là hết dữ liệu, chuyển sang khâu writer; hai, nếu chưa hết, gọi webClient.get().uri(...) — tạo request REST tới jsonplaceholder với ?page=N&size=M, lấy response JSON, dùng ObjectMapper của Jackson 3 map từng phần tử thành UserDTO; ba, tăng page += 1 rồi trả về cả list. Tức là mỗi lần read() trả về một trang user, không phải một user lẻ.

Tiếp theo, hàm processor() trong cùng file UserData.java trả về new UserProcessor(), file src/main/java/com/ecom/winners/transformers/UserProcessor.java. Class này implement ItemProcessor<List<UserDTO>, List<User>> — nhận vào cả list DTO, trả ra list entity. Bên trong có hàm process(List<UserDTO> items): nó lặp qua từng UserDTO và áp hàm lambda mapDtoToEntity. Lambda này tạo User, đồng thời tạo ba entity con: Address, GeoLocation, Company, map từng field street, city, zipcode, latitude, longitude, tên công ty... từ DTO sang entity. Đây đúng khâu "kiểm tra và dán nhãn" trên dây chuyền.

Cuối dây chuyền là hàm writer(...) trong UserData.java. Đây là lambda upsert tự viết: với từng User trong chunk, nó gọi repository.findByUserId(...) — nếu chưa có thì tạo mới, nếu có rồi thì cập nhật name, phone, address, company... rồi repository.save(user). Vì ghi theo khóa nên chạy lại bao nhiêu lần cũng không sinh dữ liệu trùng — chính là điều kiện restart an toàn mà mình nói ở slide trước.

Và hàm lắp dây chuyền lại với nhau: fetchUserData(...) trong UserData.java, dùng new StepBuilder("fetchUserData", jobRepository).<List<UserDTO>, List<User>>chunk(chunkSize).reader(...).processor(...).writer(...).faultTolerant().s
Chunk size đọc từ properties. Dòng .faultTolerant().skip(...) nghĩa là nếu một trang lỗi, Spring Batch bỏ qua và ghi nhật ký, tối đa 10 lần, không làm sập cả job.

Bước 4 — Step 2, CSV → DB: file src/main/java/com/ecom/winners/configurations/UserTransacti
Reader là FlatFileItemReader lắp bằng FlatFileItemReaderBuilder, đọc file CSV trong src/main/resources, tách cột theo header, map từng dòng thành TransactionDTO. Processor là TransactionProcessor trong file src/main/java/com/ecom/winners/transformers/Transact
hàm process(...) gọi userRepository.findByUserId(transactionDTO.getUser_id()) — nếu không tìm thấy user nào khớp thì trả null, tức là dòng đó bị bỏ qua và đếm vào skipCount; nếu tìm thấy thì tạo entity Transaction, gán user và số tiền. Writer là JpaItemWriter dựng bằng JpaItemWriterBuilder — bean có sẵn của Spring Batch, ghi cả chunk bằng một batch JPA. Tất cả được insertUserTransactions(...) trong cùng file bọc lại thành Step tương tự.

Bước 5 — Step 3, DB → Kafka: file `src/main/java/com/ecom/winners/configurations/SelectWinner` Reader ở đây đặc biệt: là một anonymous `ItemReader<User>` viết ngay trong hàm `reader(...)`, cũng `@StepScope`. Hàm `read()` của nó chỉ chạy một lần duy nhất nhờ cờ done: nó mở `EntityManager`, chạy một native query SQL chọn ngẫu nhiên một user thỏa điều kiện `SUM(amount) > :threshold` trong 7 ngày gần nhất — threshold đọc từ properties, hiện là 100. Query trả về `user_id`, rồi nó load entity `User` đầy đủ bằng JPQL. Rồi `WinnerProcessor` trong `src/main/java/com/ecom/winners/transformers/WinnerProcessor.java` biến entity thành `UserDTO`, và `KafkaItemWriter` lắp bằng `KafkaItemWriterBuilder` bắn DTO lên topic `lucky_winner` qua `KafkaTemplate`. Tên topic và template đều cấu hình trong `KafkaConfig.java`.

Vậy luồng gọi hàm hoàn chỉnh là: `JobScheduler.runSelectingWeeklyWinnerJob()` → `JobOperator.start()` → `BatchConfiguration.selectingWeeklyWinner()` chạy 3 step theo chuỗi → từng step gọi cặp reader.read() / processor.process() / writer.write() theo chunk. Ba Processor của dự án nằm trong package `transformers`, và còn hai người "gác cổng" trong package `notifiers`: `JobCompletionNotificationListener` in log khi job xong, và `SkipLoggingListener` ghi lại từng bản ghi bị skip — đúng cái "mù thông tin" mà mình nói ở đầu, giờ đã có người nhìn hộ.

Slide 15 — Khởi chạy Job và đọc nhật ký

Có ba cách khởi chạy job: bằng Java qua `JobOperator`, bằng dòng lệnh qua `CommandLine-JobRunner`, hoặc theo lịch.

Dự án demo dùng cả hai đường đầu. Đường tự động: `JobScheduler.java` khai báo annotation `@Scheduled(cron = "30 1 0 * * *", zone = ...)` — hằng ngày lúc 00:01:30 theo múi giờ cấu hình trong `.env`, nó gọi `JobOperator` để chạy job. Đường thủ công: chính là cái nút trên dashboard, lát nữa mình nói kỹ ở phần demo.

Còn đọc nhật ký thì chuẩn Spring Batch: "Executing step", "Step executed in bao nhiêu ms", "Job completed with status COMPLETED". Cái này trong log của dự án bây giờ vẫn in ra y như vậy, anh em xem terminal là thấy.

Slide 16 — Lưu đồ chống chạy trùng và Restart

Đây là phần tinh tế nhất: công thức định danh JobInstance bằng JobName cộng JobKey, tức là tên job cộng bộ tham số.

Có ba nhánh: nếu cặp này chưa từng có, chạy mới từ đầu. Nếu lần trước FAILED, truyền lại đúng tham số cũ sẽ khôi phục context để chạy tiếp. Nếu đã COMPLETED mà chạy lại cùng tham số, hệ thống ném lỗi ngay để tránh cộng tiền hai lần.

Dự án demo xử lý vấn đề này rất thực tế: job muốn chạy lại mỗi tuần, mỗi lần bấm nút, nhưng job COMPLETED rồi sẽ bị chặn. Nên trong `DashboardController`, mỗi lần khởi chạy mình truyền tham số `JobID` là một UUID ngẫu nhiên. Nhờ vậy mỗi lần bấm là một JobInstance mới, hợp lệ để chạy lại, mà lịch sử từng lần vẫn tách bạch rõ ràng trong metadata. Chống chạy trùng thì xử lý ở tầng trên: controller hỏi `JobRepository` bằng hàm `findRunningJobExecutions` xem có execution nào đang chạy không, nếu có thì trả lỗi 409, không khởi chạy thêm.

Slide 17 — Mã nguồn triển khai thực tế

Đoạn code mẫu trên slide với `.chunk(20, tx)` và `.faultTolerant().skip(...)` chính là khuôn mà dự án bám theo — các bạn thấy y hệt trong hàm `fetchUserData(...)` của `UserData.java` và `insertUserTransactions(...)` của `UserTransaction.java` ở slide trước.

Muốn đọc lại luồng gọi hàm sau buổi này thì đi theo đúng thứ tự năm file này, cũng là thứ tự các hàm được gọi khi job chạy: một, `configurations/JobScheduler.java` — hàm `runSelectingWeeklyWinnerJob()` là điểm bấm máy. Hai, `configurations/BatchConfiguration.java` — hàm `selectingWeeklyWinner()` lắp ba step. Ba, `configurations/UserData.java` — step một, API → DB, kèm `readers/RestApiItemReaderBuilder.java` và `transformers/UserProcessor.java`. Bốn, `configurations/UserTransaction.java` — step hai, CSV → DB, kèm `transformers/TransactionProcessor.java`. Năm, `configurations/SelectWinner.java` — step ba, DB → Kafka, kèm `transformers/WinnerProcessor.java`. JobRepository tự ghi lại sáu bảng BATCH_ sau mỗi commit.

Slide 18 — Demo: nút "Chọn ngay" làm gì?

Giờ là phần mọi người chờ: bấm cái nút trên giao diện thì thực sự xảy ra chuyện gì? Mình đi từng tầng một.

Trước hết, giao diện là một app React nằm trong thư mục `frontend`, được viết lại thành một trang đơn giản phục vụ khách cuối: một thẻ trạng thái, hai tab, một nút.

Khi bấm nút "Chọn ngay", file `main.jsx` sẽ gọi hàm `startJob`. Hàm này gửi một request `POST /api/jobs/executions` với body rỗng. Vite dev server chặn request có prefix `/api` và chuyển tiếp về backend ở cổng 8080 — cấu hình này nằm trong `vite.config.js`, cổng backend được đọc trực tiếp từ file `.env` ở thư mục gốc.

Về phía backend, request đó rơi vào `DashboardController.java` trong package `api`, đúng vào hàm `start()`. Hàm này chạy trên một thread boundedElastic, làm ba việc: thứ nhất, hỏi `JobRepository` — phương thức `findRunningJobExecutions` — xem job `selectingWeeklyWinner` có execution nào đang chạy không, nếu có thì ném lỗi 409 và giao diện hiện "Job đang chạy rồi". Thứ hai, tạo bộ tham số chứa `JobID` là UUID ngẫu nhiên — chính là cái nhánh "chưa có instance" trong lưu đồ slide trước. Thứ ba, gọi `operator.start(JOB_NAME, parameters)` — JobOperator của Spring Batch được Spring Boot tự động cung cấp nhờ starter batch-jdbc. Lúc này mọi chuyện tiếp diễn đúng như luồng mình đã mổ xẻ ở slide 14: `JobOperator` gọi tới bean Job trong `BatchConfiguration.java`, job chạy ba step `fetchUserData` → `insertUserTransactions` → `selectLuckyWinner`, mỗi step gọi cặp reader-processor-writer theo chunk. Step cuối chọn được winner, ví dụ user số 8, rồi `KafkaItemWriter` bắn một JSON lên topic `lucky_winner`.

Và điểm thú vị cuối cùng: winner bắn lên Kafka không bay vào khoảng không. Trong dự án mình có thêm class `WinnerRecorder.java`, một Kafka consumer nghe đúng topic đó. Mỗi message đến, nó parse JSON thành DTO và lưu vào bảng `winners` trong PostgreSQL. Dashboard cứ 5 giây gọi `GET /api/winners`, và khi thấy một người thắng mới xuất hiện, giao diện tự bật một dialog chúc mừng kèm thông tin người đó, cùng danh sách 10 người thắng gần nhất.

Vậy từ một cú click chuột, ta đi trọn vòng: React, qua HTTP vào controller, JobOperator khởi chạy job, ba step batch xử lý theo chunk, kết quả lên Kafka, consumer ghi database, rồi quay lại dashboard hiện dialog. Tất cả các bước giữa đều được JobRepository ghi vết, nên lúc demo các bạn mở tab "Lịch sử" là thấy đầy đủ từng step với số đọc, số ghi, số skip.

Slide 19 — Khi nào nên dùng, khi nào không nên

Tóm lại phần chọn dụng cụ. Spring Batch nên dùng khi: tính toán định kỳ hàng triệu bản ghi như tính lãi, tính lương; gửi email hàng loạt; tổng hợp báo cáo cuối ngày; đồng bộ dữ liệu giữa các hệ thống; hoặc như dự án này, chọn người trúng thưởng định kỳ theo tiêu chí.

Không nên dùng khi: cần trả lời ngay khi người dùng bấm nút, ví dụ nhắn tin đăng nhập; xử lý luồng sự kiện thời gian thực, đó là việc của Kafka streaming; hoặc việc nhỏ vài trăm bản ghi chỉ cần một tác vụ định kỳ đơn giản.

Quy tắc ngón tay cái: dữ liệu lớn, chạy định kỳ, không cần phản hồi tức thì, cần chịu lỗi và theo dõi chặt chẽ — bốn điều này hội đủ thì nghĩ tới Spring Batch.

Slide 20 — Tóm tắt

Kết lại, năm điều cần nhớ về Spring Batch.

Một, Job và Step: cấu trúc nhà máy, Job gồm nhiều Step chạy nối tiếp theo thứ tự an toàn. Hai, ba khâu chuẩn Reader, Processor, Writer: đọc bản ghi, xử lý biến đổi, ghi theo lô. Ba, Chunk: chia nhỏ dữ liệu, commit từng xấp chắc chắn. Bốn, JobRepository: sổ ghi chép tự động lưu trạng thái và số liệu vào database — và như demo đã thấy, phần lịch sử chạy trên dashboard chính là đọc từ đó. Năm, chịu lỗi: Skip bỏ qua bản ghi hỏng, Restart chạy tiếp khi gặp sự cố, và điều kiện để restart an toàn là ghi theo kiểu cập nhật.

Dự án demo mình vừa chạy qua chính là cả năm điều này gói trong một job ba bước, một bảng metadata tự sinh, và một cái nút trên giao diện. Cảm ơn mọi người đã lắng nghe, giờ mình mời các bạn đặt câu hỏi.